



S1 Consensus Testing Summary

Verification Evidence for the S1 Testing-Mode Implementation

Rincoin Community Forge

2026-08-26

Table of contents

1 Purpose of This Document	2
2 What Changes at H1, Briefly	2
3 Summary	3
4 Methods: What Was Tested, and How	4
4.1 Unit-level	4
4.2 Single-node functional	5
4.3 Multi-node functional	5
4.4 Cross-implementation	5
5 Results: Did It Pass?	5
5.1 Unit-level — pass	6
5.2 Single-node functional — pass	7
5.3 Multi-node functional — pass	7
5.4 Cross-implementation — partially done, passing so far	7
6 Scope: What This Testing Does Not Cover	8
7 Provenance	8

 Discussion Draft

Document version 0.1 · 2026-08-26

No scenario has been selected, preferred, or adopted. This document reports testing evidence for a **testing-mode-only** implementation of the S1 candidate; it does not argue for S1 over S5/b or S6/b, and it is not a readiness or deployment claim.

 Editorial note — for maintainers, not part of the reported evidence

This document has outstanding work before it is final: the cross-implementation producer × validator matrix (§4.4/§5.4) is not yet built, and a small number of individual test cases are still outstanding. Every one of these is called out explicitly, in its own place, below — none of the results reported as done are provisional. The full, current list is [S1/open-items.md](#).

1 Purpose of This Document

This is a management summary, not a specification. It reports whether the S1 testing-mode implementation actually behaves as `technology/consensus-transition.md` and `analysis/Rincoin_840k_S1_Consensus_Change_Specification.qmd` say it should — checked by running it, not by re-reading the design. Derivations and the normative rule itself live in those documents; this one only reports what was run, against what, and what happened. The full evidence — every test case, raw logs, exact commits — is in [S1/](#), not repeated here.

Two things this document is trying to establish, and only these two:

1. **The implementation withstands real conditions**, not just its own test suite in isolation — including other, differently-behaving node software it actually meets on a shared network.
2. **A technically skilled reader can verify the first claim without reading the design documents, the source code, or the CI logs themselves** — this document plus its annex is sufficient on its own.

The rest of the document is in three parts: **Summary** gives the outcome in a few lines; **Methods** describes what was tested and how, grouped the same way throughout; **Results** reports, group by group, whether it passed.

2 What Changes at H1, Briefly

At block height 840,000 (mainnet), three things change together: the block subsidy schedule (recursive floor(previous × 19/20) per 210,000-block epoch, no floor, no tail), a scheduled coinbase commitment identifying the chosen schedule, and a sighash-level replay identifier (`sig_fork_id`) mixed into new transaction signatures. Full rationale

and byte-level specification: `technology/consensus-transition.md`. This document assumes that design and tests it; it does not re-argue it.

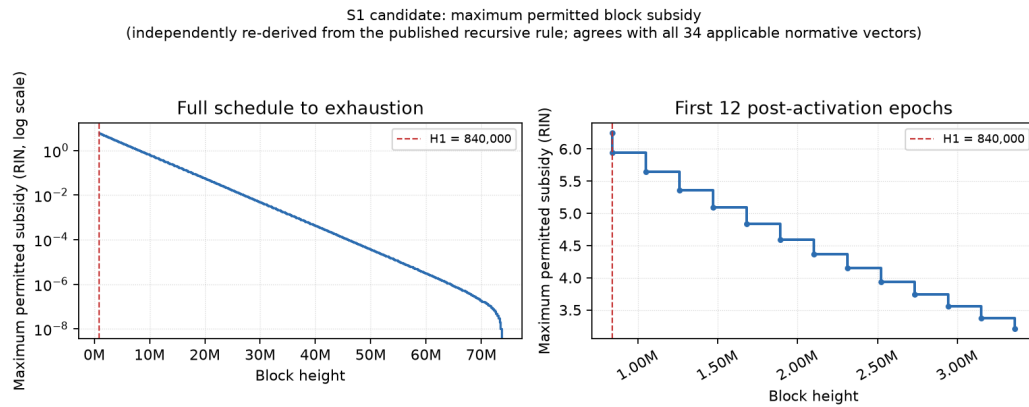


Figure 1: Maximum permitted block subsidy under the S1 candidate, independently re-derived from the published recursive rule (not imported from the implementation under test) and confirmed to agree with all 34 applicable rows of `analysis/data/S1_normative_test_vectors.csv`, the specification’s own conformance vector file.

3 Summary

Every test that has been run passes. 532 of 532 C++ unit tests, 159 of 159 functional tests (the fork-specific suite plus the full pre-existing CI allowlist, unmodified), and every cross-implementation comparison run so far. Two real defects were found and fixed while building this evidence, reported in Results as part of it, not as a caveat against it.

Evidence layer	Result	Status
C++ unit tests	532 / 532 pass	Done
Fork scenario functional tests (single- and two-node)	6 / 6 pass	Done
Full functional CI allowlist (fork + pre-existing, unmodified)	159 / 159 pass	Done
Independent (non-C++) subsidy re-derivation	agrees with all 34 applicable vectors	Done
Cross-implementation producer x validator matrix	not yet run	Outstanding
Fuzz targets (commitment parsing, <code>sig_fork_id</code> sighash mixing)	not yet written	Outstanding

Evidence layer	Result	Status
Pool / seeder integration rehearsal	not yet run	Outstanding

One evidence layer is not yet complete: the full cross-implementation producer $\times$ validator matrix (comparing this implementation against older Rincoin releases and independently developed forks, in both directions, run by a reusable tool rather than the two narrower hand-written comparisons currently in place). Two related, narrower comparisons that *are* done are reported in Results §5.4, including a real finding about a currently-released Rincoin version’s actual behavior at the fork height.

Read **Methods** below for what “tested” means in each row above and why it was tested that way; read **Results** for the detail behind each pass.

4 Methods: What Was Tested, and How

Four groups, used consistently through both this section and Results:

Group	What it checks	Why it’s a separate group
Unit-level	Each piece of consensus logic (subsidy formula, commitment parsing, sighash mixing) against frozen, byte-exact vectors	Smallest testable unit; catches an error before it can reach anything that depends on it
Single-node functional	The same rules, enforced end-to-end by a real running node via RPC/P2P	Unit tests exercise logic in-process; this confirms the rules hold when driven the way a real deployment would drive them
Multi-node functional	Whether the network prefers higher work only when that chain is also rule-valid	This is the entire reason the coinbase-commitment mechanism exists — it has to be checked directly, not inferred from the rule existing
Cross-implementation	What happens when this build meets software that disagrees with it	The only layer that reflects the network as it actually is, not just this codebase

4.1 Unit-level

The subsidy formula, the coinbase-commitment builder/parser, and the `sig_fork_id` sighash-mixing code are each checked in isolation against frozen vectors — values fixed in advance, not computed by calling the code under test. For the subsidy formula specifically, “frozen” means two independent sources agreeing:

`analysis/data/S1_normative_test_vectors.csv` (the file the specification itself names as the conformance target) and a from-scratch Python re-derivation of the published recursive rule (`verification/scripts/independently_derive_s1_schedule.py`) that does not import the C++ implementation, mirroring `analysis/scripts/verify_s6b_independently`. existing role for S6/b.

Every pre-existing sighash and script unit test in the codebase is also re-run, unmodified, as a no-regression guard — confirming the new sighash mixing produces byte-identical output for every caller that doesn't opt into it.

4.2 Single-node functional

Driven through a real, running `rincoind` process via its actual RPC and P2P interfaces (the same `TestNode` framework `analysis`'s own reproducibility scripts assume readers are comfortable with), not exercised in-process: coinbase-commitment validation (every malformation in the vector catalog, see [S1/commitment-test-vectors.md](#)), subsidy enforcement at and around the activation boundary, and `sig_fork_id` transaction propagation and rejection before and after activation.

4.3 Multi-node functional

Two testing-mode nodes, connected and synced past the activation height, one isolated and used to mine two competing continuations: a longer chain that is also commitment-valid, and a longer chain that is commitment-*invalid*. Both chains are offered back to the network; which one is adopted is the entire test.

4.4 Cross-implementation

Every accessible node binary — this implementation, older Rincoin releases, and independently developed forks reachable on a shared network — tested as both block *producer* and block *validator* against every other binary, with expected outcomes taken from the frozen vectors above, never from any implementation's own output (so no single binary is treated as ground truth).

i Editorial note — for maintainers

The general-purpose tool that runs this comparison for an arbitrary set of binaries (`test/fork_interop_matrix.py`, so the same tool serves S5/b and S6/b later without being rewritten) is not yet built. Two narrower, hand-written comparisons currently stand in for it — `feature_fork_vs_legacy.py` and `feature_fork_vs_aevust.py` — and their results are reported in Results §5.4.

5 Results: Did It Pass?

Same four groups as Methods, each reporting a concrete outcome, not a restated description of the test.

5.1 Unit-level — pass

532 of 532 C++ unit tests pass. (One unrelated, pre-existing warning: a missing test-data environment variable skips a single unit test; not a failure, not introduced by this work.)

The subsidy schedule’s two independent sources — the specification’s own normative vector file and the from-scratch Python re-derivation — agree with each other and with the C++ implementation at every height checked:

Height	Vector ID	CSV value (base units)	C++ unit test result	Note
839,999	S1-001	625,000,000	625,000,000 — match	last pre-activation block
840,000	S1-002	593,750,000	593,750,000 — match	activation block; first block of post-activation epoch 1
2,729,999	S1-020	393,905,878	393,905,878 — match	last block of post-activation epoch 9
2,730,000	S1-021	374,210,584	374,210,584 — match	first block of post-activation epoch 10

The commitment-parsing vector catalog is covered with two specific exceptions, tracked individually rather than folded into a round number: see [S1/commitment-test-vectors.md](#) for exactly which two and why.

This is also the layer where both real defects mentioned in the Summary were actually found — both by unit tests written for this work, not by code review:

- A wallet operation crashed the whole test process (not just failed the one operation) when exercised on a wallet that hadn’t yet processed a block, via an overly strict internal assertion newly reached by this branch’s fork-context lookup. First visible as 91 unrelated-looking failures in a completely different part of the test suite; traced with a debugger to this root cause.
- The component that produces `sig_fork_id`-aware signatures had a latent bug that made its own internal self-check reject its own correctly created signatures — a real defect on the PSBT-signing path (confirmed against the actual call sites, not a test-only artifact), caught by a unit test written specifically to exercise that path directly rather than through a wrapper that happened to route around the bug.

Full detail and reproduction commands: [S1/unit-test-results.md](#).

5.2 Single-node functional — pass

All commitment-validation, subsidy-enforcement, and `sig_fork_id` checks pass against a real running node, across every boundary and malformation case tested. Part of a 159-test functional run that also re-runs every pre-existing (non-fork-specific) functional test unmodified, confirming no regression to ordinary wallet, mempool, P2P, mining, or MWEB behavior.

5.3 Multi-node functional — pass

Both reorg outcomes are correct: the higher-work, commitment-*valid* chain is adopted; the higher-work, commitment-*invalid* chain is not — confirming the mechanism does what it exists to do, not just that a valid chain is accepted.

`-reindex-chainstate` was additionally checked empirically across the activation height (mine past H1, restart with `-reindex-chainstate`, compare tip and `verifychain`): identical tip, clean result. This does not by itself re-run block-acceptance-time consensus checks — a pre-existing, long-standing property shared by every other height-gated consensus rule already in this codebase, not something specific to this branch — so it's reported as a confirmation that behavior matches that established property, not as an independent re-validation.

5.4 Cross-implementation — partially done, passing so far

The full producer $\times$ validator matrix has not been run (see the Methods editorial note). Two narrower comparisons have, and both pass for what they cover:

- **S1 vs. v1.1.0** (`rincoin-community/rincoin-core`): clean divergence at the activation height — this implementation applies its rules, v1.1.0 does not adopt it. A genuine finding surfaced along the way: v1.1.0 itself does **not** halt or refuse to extend the chain past height 840,000 on a non-mainnet chain, contradicting that release's own stated branch policy (whose halt height turned out to be wired for mainnet only). Reported as a finding about that release's actual, directly observed behavior, not about S1.
- **S1 vs. v1.0.1** (`Rin-coin/rincoin`, no fork awareness at all): clean divergence, no crash, no accidental adoption of the S1 chain — confirmed directly via `getbestblockhash` on both nodes.
- **S1 vs. Aevust/rincoin@feature/port-sim-v1.0.7** (an independently developed, foreign fork): the foreign branch's headers can become this node's best-known-header, but full-block validation correctly rejects it at H1 for a missing/wrong commitment — the validated tip (`getbestblockhash`) never adopts it, even while header-tracking RPCs see it.

Full detail: [S1/functional-test-results.md](#).

6 Scope: What This Testing Does Not Cover

Deliberate scoping, not oversight:

- **Header-level branch isolation** (a block-header version bit as a second, independent identity mechanism) — designed and then dropped from the active proposal; kept only as a fully specified, inactive contingency. Not part of what S1 actually implements, so not tested.
- **A dedicated P2P branch-announcement message** — no such message exists in the current design (commitment-only + `sig_fork_id`); an earlier, since-abandoned draft used one.
- **Staged release engineering** (build profiles, signalling windows, cross-party coordination gates) — production-deployment planning for *after* a scenario is selected. Nothing is selected yet. This implementation is a single testing-mode build that refuses to run on mainnet at all without an explicit environment variable set at every invocation; that is the right level of complexity for what it currently is.
- **Live mainnet telemetry or monitoring** — not applicable; this build cannot run on mainnet.
- **Pool- and DNS-seeder-software integration rehearsals** — real-world integration points worth covering (a pool rebuilding the coinbase transaction is exactly the kind of place a commitment silently gets stripped), tracked as outstanding rather than silently skipped: [S1/open-items.md](#).

7 Provenance

Repository	<code>rincoin-community/rincoin-core</code>
Branch	<code>consensus/s1-testing</code>
Commit tested	<code>73eb5b3ce</code>
CI run	github.com/rincoin-community/rincoin-core/actions/runs/33994483614 — concluded, success (all three jobs green; unit+functional's first attempt hit a shared <code>feature_taproot.py</code> CI-runner flake, confirmed transient by a clean rerun)
branch_id	<code>00112233445566778899aabbccddeeff</code> — the specification's own canonical synthetic test value (<code>technology/consensus-transition.md</code> §5), migrated from an earlier ad-hoc value on this same commit
Build	testing-mode only; refuses to start on mainnet unless <code>RINCOIN_TESTING_ALLOW_MAINNET=1</code> is set in the process environment

8 Where to Go Next

- Full evidence, raw results, exact reproduction commands: [S1/](#)
- The mechanism being tested: [technology/consensus-transition.md](#),
[technology/replay-protection-plan.md](#)
- The normative rule and its own vector set: [analysis/Rincoin_840k_S1_Consensus_Change_Specificity.md](#)
- Outstanding work tracked for maintainers: [S1/open-items.md](#)

DRAFT